

01 Hive Theory - Basics -KirkYagami

- Hive is a data warehousing tool that provides an SQL-like interface to query data stored in Hadoop.
- ETL tool for hadoop ecosystem.
- Is a SQL like querying tool to query the data stored in HDFS and other Filesystems that integrate with Hadoop.
- Developed by Facebook and later on taken by Apache.
- It processes Structured data that can be stored into Tables.
- Efficient for Batch Processing.
- Hive is a lens between MapReduce and HDFS.
- It provides us various storage file formats like Parquet, Sequence file, ORC file, Text file with significant compression.
- Provides Beeline client which is used to connect from Java, Scala, C#, Python, and many more languages.

What hive is not?

- Not a database.
- Not an OLTP tool. Closer to being OLAP tool.
- Does not provide row level insert, update and delete. (- Update is only possible for ORC format.)

WHY?

- To reduce the complexity of MapReduce Programming, Mappers, Reducers and Driver code
- 100s of lines of MR code can be summarized into one line query.
- Analysts can use SQL knowledge, No need of Java.

Hive VS SQL

Hive	SQL
Data warehouse project for data analysis, built on top of Hadoop. Does not store physical data	Query language for RDMS
Write once and Read-Many times	Write-Many and Read-Many
OLAP systems	OLTP systems
Highly scalable at lowcost	Not easily scalable

Hive VS RDBMS

Hive	RDBMS
Data warehouse on top of Hadoop. Geared towards Analytical Processing.	RDBMS are for transactional processing
Batch Processing	Transactional Processing
1. Optimal for processing large datasets of data. Gigabytes/Petabytes.	1. OLTP systems are optimal for handling a large number of short online transactions, like insert, update, and delete operations.
2. Data in Hive is meant to be used for Analytical Purposes. For example: To process orders data for trends in the last 3 years.	2. Data in RDBMS is meant to be used for real-time transaction processing.
3. Since Hive uses Hadoop and MapReduce, it leverages the tremendous parallel computing power of these frameworks.	3. RDBMS handles transactions efficiently but is not designed for massive parallel processing.
4. Hive/Hadoop uses a large number of cheap machines in parallel. Performance is linearly increased when more machines are added.	4. RDBMS improves performance by using indexing. Building indexes takes time and a lot of disk space, which can be expensive. Performance does not scale linearly with increasing disk space.
5. Not for transactional processing. Even fetching a single row will launch a MapReduce job that might take a few minutes to run.	5. A well-designed RDBMS can answer transactional queries in milliseconds or microseconds.
6. Hive does not support row-level updates.	6. RDBMS supports row-level updates and deletes.
7. Once the data is written to Hive, its purpose is read-only. Analysts would only read historical data, never update it.	7. RDBMS allows data to be frequently updated and modified.
Schema-on-Read	Schema-on-Write
Schema: Description of a database table: column names, column types, constraints.	Schema: Description of a database table: column names, column types, constraints, enforced at the time of data write.
Data of the table is stored in files in HDFS. Hive is not the owner of files. Data can be modified by other clients. For example: The same files might be shared between Hive, HBase, Cassandra. Because underlying files can be changed at any time, Hive cannot enforce Schema-on-Write. Hive metastore has the instructions for reading and parsing the HDFS files.	Complete control over data, no external program can access the data without going through the database.

Hive	RDBMS
During load/insert operations, Hive will just dump the data into a file without checking the schema. Since it is unaware of schema during write operations, you cannot do row-level updates/deletes.	During load/insert operations, RDBMS checks the data against the schema, ensuring data integrity and consistency.
- Hive has extensive support for joins. - Hive does not support Natural Joins.	

```
select a.firstname, b.subordinate
from employees,subordinates;
```

SQL

Deleting and updating a single row in a table

Hive does not provide a way to perform these two operations!

- Hive pretends that the underlying data is in the form of a table but it is a batch processing system at heart.
- Hive has massive overheads in job submission and scheduling.
- Hive does not offer real-time queries and row-level updates.

Introduction to Hive Internal and External tables

Fundamentally, Hive knows two different types of tables: **Internal table** and the **External table**. The Internal table is also known as the managed table.

We can identify the internal or External tables using the `DESCRIBE FORMATTED table_name` statement in the Hive, which will display either `MANAGED_TABLE` or `EXTERNAL_TABLE` depending on the table type.

1. Hive Internal Table

Hive owns the data for the internal tables.

It is the default table in Hive. When the user creates a table in Hive without specifying it as external, then by default, an internal table gets created in a specific location in HDFS.

By default, an internal table will be created in a folder path similar to `/user/hive/warehouse` directory of HDFS. We can override the default location by the location property during table creation.

If we drop the managed table or partition, the table data and the metadata associated with that table will be deleted from the HDFS.

2. Hive External Table

Hive does not manage the data of the External table.

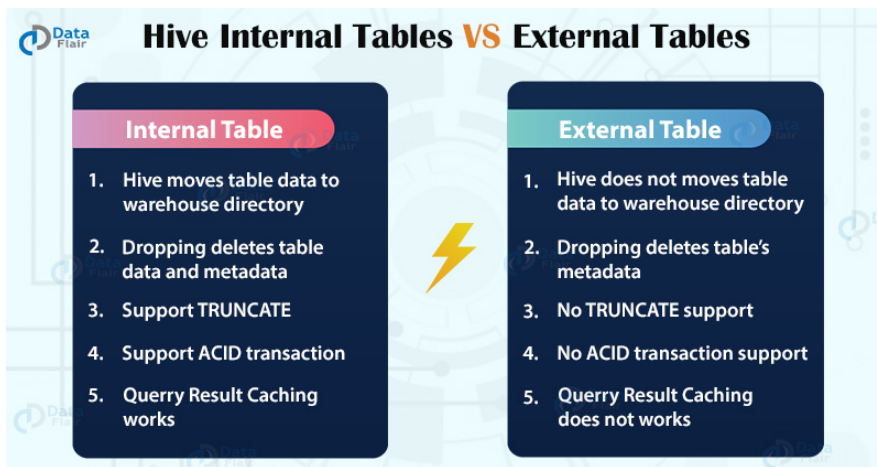
We create an external table for external use as when we want to use the data outside the Hive.

External tables are stored outside the warehouse directory. They can access data stored in sources such as remote HDFS locations or Azure Storage Volumes.

Whenever we drop the external table, then only the metadata associated with the table will get deleted, the table data remains untouched by Hive.

We can create the external table by specifying the **EXTERNAL** keyword in the Hive create table statement.

Difference between Hive Internal and External Table



Let us now see the difference between both Hive tables. The major differences in the internal and external tables in Hive are:

1. LOAD semantics

The Load semantics varies in both the tables. Let us see the difference in load semantics between the internal table and the external table.

a. Internal Table

When we load data into an internal table, then Hive moves data into the warehouse directory.

For example:

Here in this example, we are creating a table 'internaldemo'. When we load data into the 'internaldemo' table, then Hive moves the data to the warehouse directory.

```

dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
0: jdbc:hive2://localhost:10000> CREATE TABLE internaldemo (col1 STRING, col2 int);
INFO : Compiling command(queryId=dataflair_20200211150249_b877b7a5-88d3-49e6-984c-66340e066ef5): CREATE TA
BLE internaldemo (col1 STRING, col2 int)
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Semantic Analysis Completed (retrial = false)
INFO : Returning Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=dataflair_20200211150249_b877b7a5-88d3-49e6-984c-66340e066ef5);
Time taken: 0.076 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=dataflair_20200211150249_b877b7a5-88d3-49e6-984c-66340e066ef5): CREATE TA
BLE internaldemo (col1 STRING, col2 int)
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=dataflair_20200211150249_b877b7a5-88d3-49e6-984c-66340e066ef5);
Time taken: 2.081 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager
No rows affected (2.169 seconds)
0: jdbc:hive2://localhost:10000> 

```

Now, loading data into the internal table created above.

```
dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
0: jdbc:hive2://localhost:10000> LOAD DATA LOCAL INPATH '/home/dataflair/demo' INTO TABLE internaldemo;
INFO : Compiling command(queryId=dataflair_20200211151427_d781cd97-1eee-4a3e-8356-cbf080075b40): LOAD DATA
LOCAL INPATH '/home/dataflair/demo' INTO TABLE internaldemo
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Semantic Analysis Completed (retrial = false)
INFO : Returning Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=dataflair_20200211151427_d781cd97-1eee-4a3e-8356-cbf080075b40);
Time taken: 0.074 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=dataflair_20200211151427_d781cd97-1eee-4a3e-8356-cbf080075b40): LOAD DATA
LOCAL INPATH '/home/dataflair/demo' INTO TABLE internaldemo
INFO : Starting task [Stage-0:MOVE] in serial mode
INFO : Loading data to table newdataflair.internaldemo from file:/home/dataflair/demo
INFO : Starting task [Stage-1:STATS] in serial mode
INFO : Completed executing command(queryId=dataflair_20200211151427_d781cd97-1eee-4a3e-8356-cbf080075b40);
Time taken: 0.252 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager
No rows affected (0.336 seconds)
0: jdbc:hive2://localhost:10000> 
```

On describing the table, we see that the table data is moved to the Hive warehouse directory.

```
dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
|
| # Detailed Table Information | NULL | NULL
|
| Database: | newdataflair | NULL
|
| OwnerType: | USER | NULL
|
| Owner: | dataflair | NULL
|
| CreateTime: | Tue Feb 11 15:02:49 IST 2020 | NULL
|
| LastAccessTime: | UNKNOWN | NULL
|
| Retention: | 0 | NULL
|
| Location: | hdfs://localhost:9000/user/hive/warehouse/DATAFLAIR/internaldemo | NULL
|
| Table Type: | MANAGED_TABLE | NULL
|
| Table Parameters: | NULL | NULL
|
| | bucketing_version | 2
|
| | numFiles | 1
|
| | numRows | 0
|
```

b. External Table

With the EXTERNAL keyword, Hive knows that it is not managing the table data, so it does not move data to its warehouse directory. Hive does not even check whether the external location at the time it is defined exists or not.

For example:

In this example, we are creating an external table, 'external_demo' on the specified location that is '/home/dataflair/'. On loading the data into the external table, the Hive does not move table data to its warehouse directory.

```

dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
0: jdbc:hive2://localhost:10000> CREATE EXTERNAL TABLE external_demo(col1 STRING, col2 STRING) LOCATION '/home/dataflair/';
INFO : Compiling command(queryId=dataflair_20200211153144_ec22d9b6-67a6-49a5-b3ec-27af33eb71d8): CREATE EXTERNAL TABLE external_demo(col1 STRING, col2 STRING) LOCATION '/home/dataflair/'
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Semantic Analysis Completed (retrial = false)
INFO : Returning Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=dataflair_20200211153144_ec22d9b6-67a6-49a5-b3ec-27af33eb71d8);
Time taken: 0.061 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=dataflair_20200211153144_ec22d9b6-67a6-49a5-b3ec-27af33eb71d8): CREATE EXTERNAL TABLE external_demo(col1 STRING, col2 STRING) LOCATION '/home/dataflair/'
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=dataflair_20200211153144_ec22d9b6-67a6-49a5-b3ec-27af33eb71d8);
Time taken: 0.109 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager
No rows affected (0.208 seconds)
0: jdbc:hive2://localhost:10000> 

```

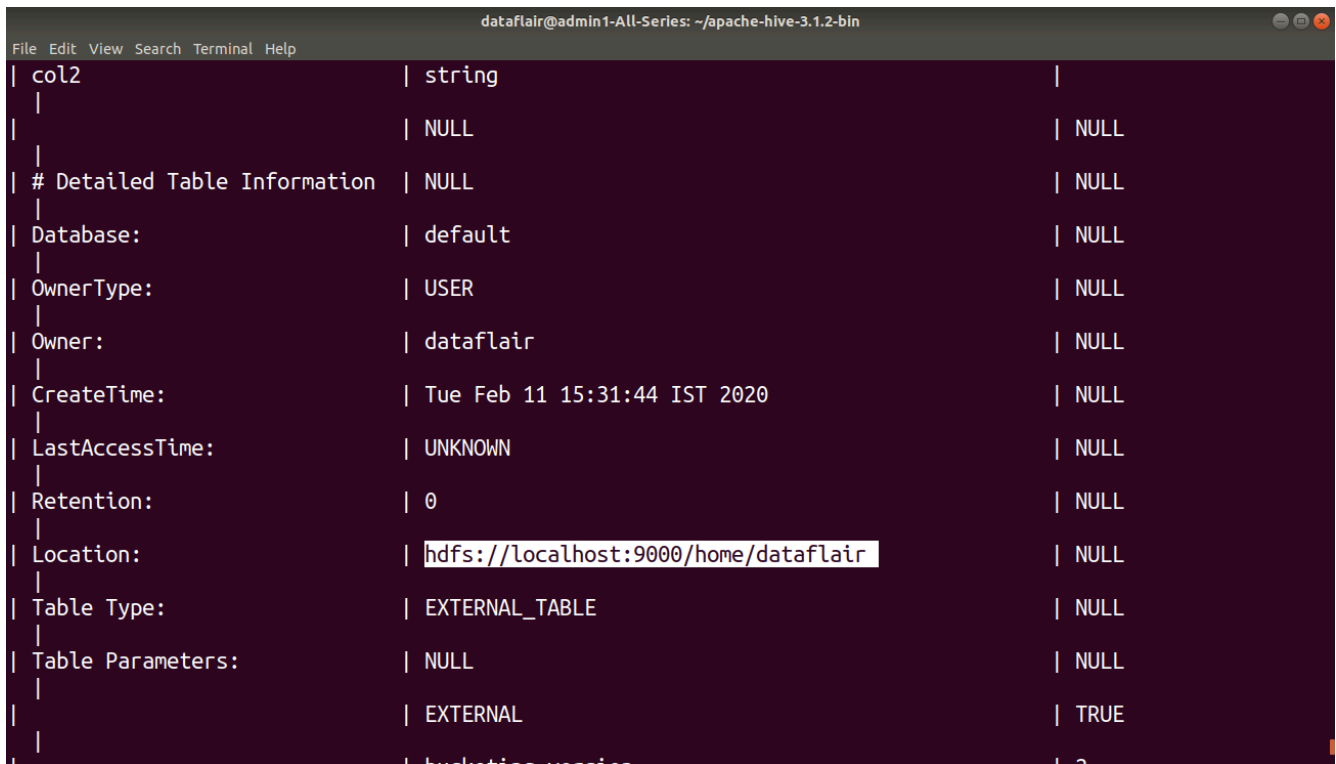
Now, loading data to the external table created above.

```

dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
0: jdbc:hive2://localhost:10000> LOAD DATA LOCAL INPATH '/home/dataflair/demo' INTO TABLE external_demo;
INFO : Compiling command(queryId=dataflair_20200211153258_baa8dd23-6337-430c-8265-8e8a8c038563): LOAD DATA LOCAL INPATH '/home/dataflair/demo' INTO TABLE external_demo
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Semantic Analysis Completed (retrial = false)
INFO : Returning Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=dataflair_20200211153258_baa8dd23-6337-430c-8265-8e8a8c038563);
Time taken: 0.023 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=dataflair_20200211153258_baa8dd23-6337-430c-8265-8e8a8c038563): LOAD DATA LOCAL INPATH '/home/dataflair/demo' INTO TABLE external_demo
INFO : Starting task [Stage-0:MOVE] in serial mode
INFO : Loading data to table default.external_demo from file:/home/dataflair/demo
INFO : Starting task [Stage-1:STATS] in serial mode
INFO : Completed executing command(queryId=dataflair_20200211153258_baa8dd23-6337-430c-8265-8e8a8c038563);
Time taken: 0.172 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager
No rows affected (0.207 seconds)
0: jdbc:hive2://localhost:10000> 

```

On browsing the table, we can see that the Hive table data is not moved to the Hive warehouse directory. It is stored in the location specified while creating a table.



```

dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
| col2 | string |
| | NULL | NULL
| # Detailed Table Information | NULL | NULL
| Database: | default | NULL
| OwnerType: | USER | NULL
| Owner: | dataflair | NULL
| CreateTime: | Tue Feb 11 15:31:44 IST 2020 | NULL
| LastAccessTime: | UNKNOWN | NULL
| Retention: | 0 | NULL
| Location: | hdfs://localhost:9000/home/dataflair | NULL
| Table Type: | EXTERNAL_TABLE | NULL
| Table Parameters: | NULL | NULL
| | EXTERNAL | TRUE

```

2. DROP semantics

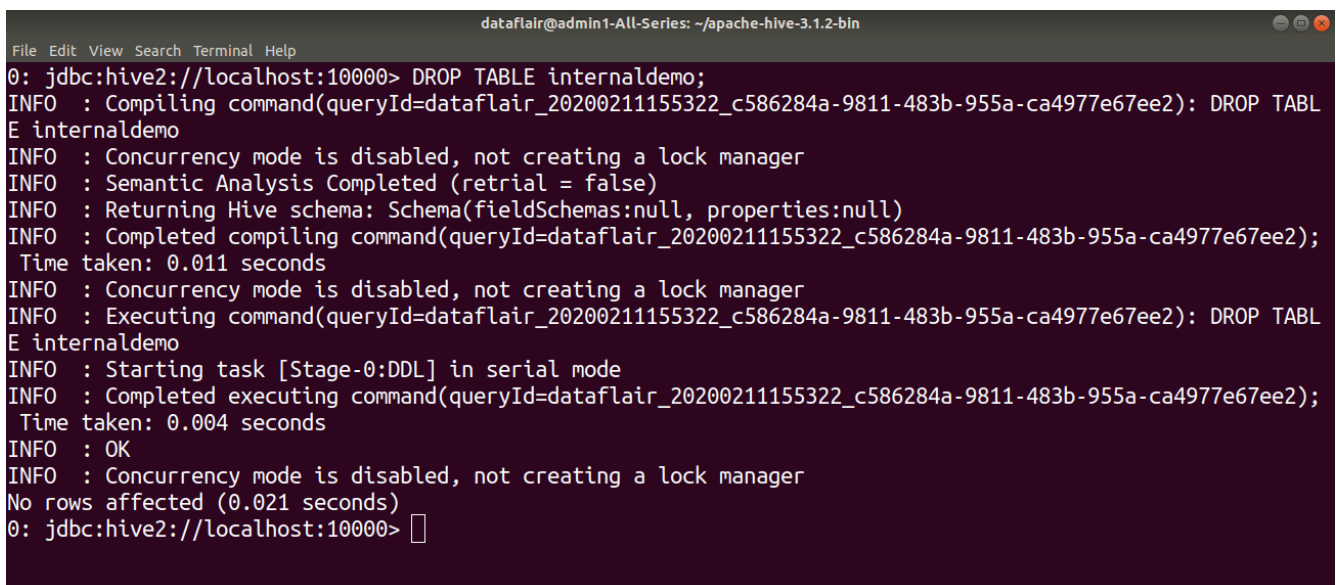
Like load semantics, drop semantics also varies in both tables. Let us see the difference in drop semantics between the internal table and the external table.

a. Internal table

Dropping the internal table will delete the table data, as well as the metadata associated with the table.

For example:

In this example, we are dropping the managed table 'internaldemo'. You can see that it will delete both the table metadata as well as the table data.



```

dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
0: jdbc:hive2://localhost:10000> DROP TABLE internaldemo;
INFO : Compiling command(queryId=dataflair_20200211155322_c586284a-9811-483b-955a-ca4977e67ee2): DROP TABL
E internaldemo
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Semantic Analysis Completed (retrial = false)
INFO : Returning Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=dataflair_20200211155322_c586284a-9811-483b-955a-ca4977e67ee2);
Time taken: 0.011 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=dataflair_20200211155322_c586284a-9811-483b-955a-ca4977e67ee2): DROP TABL
E internaldemo
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=dataflair_20200211155322_c586284a-9811-483b-955a-ca4977e67ee2);
Time taken: 0.004 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager
No rows affected (0.021 seconds)
0: jdbc:hive2://localhost:10000> 

```



```

dataflair@admin1-All-Series: ~
File Edit View Search Terminal Help
dataflair@admin1-All-Series:~$ hadoop fs -ls /user/hive/warehouse/DATAFLAIR/internaldemo;
ls: `/user/hive/warehouse/DATAFLAIR/internaldemo': No such file or directory
dataflair@admin1-All-Series:~$

```

Table data also gets deleted from the HDFS.

b. External table

Dropping the external table will delete only table metadata. The table content remains untouched.

For example:

In this example, we can see that on deleting the external table 'external demo', the content of the table is still present in the HDFS location.

```

dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
0: jdbc:hive2://localhost:10000> DROP TABLE externaldemo;
INFO : Compiling command(queryId=dataflair_20200211164802_6d1d7373-2015-4e52-b3d7-18b4c6589cd5): DROP TABL
E externaldemo
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Semantic Analysis Completed (retrial = false)
INFO : Returning Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=dataflair_20200211164802_6d1d7373-2015-4e52-b3d7-18b4c6589cd5);
Time taken: 0.024 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=dataflair_20200211164802_6d1d7373-2015-4e52-b3d7-18b4c6589cd5): DROP TABL
E externaldemo
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=dataflair_20200211164802_6d1d7373-2015-4e52-b3d7-18b4c6589cd5);
Time taken: 0.079 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager
No rows affected (0.119 seconds)
0: jdbc:hive2://localhost:10000>

```

```

dataflair@admin1-All-Series: ~
File Edit View Search Terminal Help
dataflair@admin1-All-Series:~$ hadoop fs -ls hdfs://localhost:9000/user/hive/warehouse/newdb/externaldemo;
Found 1 items
-rw-r--r-- 1 dataflair supergroup 59 2020-02-11 16:47 hdfs://localhost:9000/user/hive/warehouse/n
ewdb/externaldemo/demo
dataflair@admin1-All-Series:~$

```

3. TRUNCATE support

The **TRUNCATE** command only works for the internal table.

For example:

In this example, we are trying to use the truncate command with 'externaldemo' table. We will get an error message that says 'Cannot truncate non-managed table externaldemo'.

```

dataflair@admin1-All-Series: ~/apache-hive-3.1.2-bin
File Edit View Search Terminal Help
0: jdbc:hive2://localhost:10000> truncate table externaldemo;
Error: Error while compiling statement: FAILED: SemanticException [Error 10146]: Cannot truncate non-manage
d table externaldemo. (state=42000,code=10146)
0: jdbc:hive2://localhost:10000>

```

4. ACID Support

ACID/transactional works only for the internal table. They do not work for External Table.

5. Query Result Caching

Query Result Caching that saves the results of an executed Hive query for reuse on subsequent queries only works for the internal table.

Thus there are some differences between the Hive internal table and the external table.

Let us now see when to use which Hive table.

When to use the Internal and External table?

1. Hive Internal Table

We can use the internal table in cases:

- When generating temporary tables.
- When required that Hive should manage the lifecycle of the table.
- And when we don't want table data after deletion.

2. Hive External Table

We can use the external table in cases:

- When we are not creating the table based on the existing table.
- When required to use data outside of Hive. For example, the data files are read and processed by an existing program that does not lock the files.
- When we don't want to delete the table data completely even after DROP.
- When the data should not be own by Hive.

So, this was all in Hive internal and external tables.